

Operating Systems

2520030273

K.Navadeep

SEC-5

WEEK-3

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Code

```
kolasani_navadeep@Navadee × + v
GNU nano 8.7.1 fork1.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
    }
    else if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());

        sleep(10);

        printf("Child finished\n");
    }
    else
    {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        sleep(5);

        wait(NULL);

        printf("Parent finished\n");
    }

    return 0;
}
```

output

```
kolasani_navadeep@Navadeep-Vitus:~$ nano fork1.c
kolasani_navadeep@Navadeep-Vitus:~$ gcc fork1.c
kolasani_navadeep@Navadeep-Vitus:~$ ./a.out
Parent Process
PID = 766
Child PID = 767
Child Process
PID = 767
PPID = 766
Child finished
Parent finished
kolasani_navadeep@Navadeep-Vitus:~$ |
```